> **Referencia en vivo y laboratorio:** para el recorrido sobre el asistente publicado y el laboratorio PostgreSQL/pgvector (`lab/`), consulta también `recorrido-cloud.md` . Este guion articula los 90 minutos de S11 sobre las 16 diapositivas de `material/presentaciones/S11-crag.pptx` .
>
> **Aviso sobre la presentación:** el PPTX no se ha actualizado con el laboratorio Zocomar. Las diapositivas **4, 13 y 15** muestran todavía el ejemplo guionizado del curso, el laboratorio AetherCloud y los números del respaldo determinista. Este guion indica en cada una qué decir y qué proyectar en su lugar. La diapositiva 12 muestra «20 s»: el asistente publicado usa 60 s con Sonnet.

Guion S11 · Corrective RAG (CRAG): reordenamiento, suficiencia, grafo y métricas

Fecha: martes 27 de octubre de 2026 · **Cohorte:** M/J · **Horario:** 18:30–20:00 `Europe/Madrid` (90 minutos con preguntas).
Presentación: `material/presentaciones/S11-crag.pptx` (`material/presentaciones/S11-crag.pdf`), 16 diapositivas en cuatro secciones: Inicio (1–3), RAG avanzado y CRAG (4–12), Demo (13) y Evaluación y métricas (14–16).
Laboratorio: `lab/` con la tienda ficticia **Zocomar** (`bun run lab:real compare` , `demo` y `eval` con modelos reales; `bun run lab:demo` y `lab:eval` como respaldo sin red).
Demostrador del curso: `bun run dev` en `http://localhost:5173/` para los seis guiones deterministas del grafo.
Evidencia de hoy: `docs/evidence/zocomar-lab-2026-10-24.md` y los recibos de `docs/evidence/zocomar-2026-10-24/` .

Temario de thePower para S11: RAG avanzado (superar las limitaciones del RAG básico con técnicas de autocorrección y reordenamiento de resultados) · Corrective RAG · Evaluación y métricas específicas de sistemas RAG. Ejercicio: construir un CRAG completo desde cero.

Reparto del tiempo: 20 min de conceptos (diapositivas 1–12), 55 min de demo (diapositiva 13: guiones locales y comparación real en Zocomar) y 15 min de resultados, ejercicio y dudas (diapositivas 14–16). El QR solo se muestra al inicio.

Mensaje de la sesión, con datos reales: con un buen generador y un corpus pequeño y limpio, el RAG básico ya acierta; CRAG acierta igual, pero con margen y con un contexto limpio, y lo paga en latencia. CRAG es un seguro, no una mejora gratis en cada respuesta.

Nombres de los nodos: el laboratorio y el enunciado usan `retrieve` , `rerank` , `assess` , `retain` , `discard` , `reformulate` , `answer` y `refuse` (`lab/src/crag-graph.ts`). En el panel de traza del asistente del curso (`apps/api/src/crag-graph.ts`) el reordenamiento aparece como `rank` .

0–3 min · Apertura y QR de S10 (Diapositivas 1 a 3)

Diapositiva 1 · CRAG: RAG avanzado, cuando una búsqueda no basta

Explica:

> En S10 construimos un RAG de una sola búsqueda con umbral previo. Hoy le añadimos una capa de decisión: un grafo de **Corrective RAG (CRAG)** en LangGraph que juzga si la evidencia cubre la pregunta y, si no, la corrige. Suficiente → responder; parcial → conservar y reformular; insuficiente

> → descartar y reformular. Todo con límites en el código. Para medirlo con modelos reales usamos un corpus nuevo, **Zocomar**, una tienda inventada.

Diapositiva 2 · Evalúa la clase anterior (QR)

Deja el QR visible un minuto y explica para qué sirve, como pidió thePower:

> Este QR es la encuesta de satisfacción de AI Engineer y evalúa nuestra clase anterior: **S10, la del jueves 22 de octubre**, no la de hoy. Es muy importante que la rellenéis: con vuestras respuestas ajustamos ritmo, profundidad y práctica.

Mientras responden, pide que recuerden una pregunta que el RAG de S10 rechazó aunque la respuesta estaba en los documentos.

Diapositiva 3 · Lo que pide el temario, en tres bloques

Presenta el temario de thePower tal como está en la ficha del programa y su reparto: conceptos (20 min), Corrective RAG en vivo (55 min: guiones locales y comparación real en Zocomar) y evaluación con resultados reales (15 min).

3–20 min · RAG avanzado y CRAG: problema, técnicas, grafo, cobertura y límites (Diapositivas 4 a 12)

Diapositiva 4 · El límite del RAG básico (usa el ensayo real de S10)

La diapositiva muestra el ejemplo guionizado de S5 y S6. **Cuenta en su lugar el caso real del ensayo de S10**, que los alumnos ya vieron con AetherCloud (`material/S10/evidencias/evaluacion-real.json`):

- Pregunta con respuesta: *«¿En cuántos segundos como máximo se propaga una revocación de emergencia a las pasarelas europeas?»*. Está en el documento 02: menos de 90 segundos.
- La búsqueda lo encontró **en primera posición**, con similitud **0,679**, por debajo del umbral calibrado **0,725**. El RAG rechazó: «no consta».
- Al revés: la pregunta por el **NIF**, que no existe, sacó **0,712**, más que la respondible.
- En ese ensayo el documento correcto salió primero en **14 de 14** preguntas; los cinco fallos (casos 4, 5, 6, 10 y 14) fueron del umbral, no de la búsqueda.

Mensaje: **la similitud mide la cercanía de tema, no si el texto contiene la respuesta.** Un umbral fijo falla en un sentido o en otro. CRAG sustituye ese corte por un juez que lee el fragmento frente a la pregunta.

Diapositiva 5 · Dónde se puede mejorar un RAG básico

Primera pieza del temario: RAG avanzado como **superar las limitaciones del RAG básico con autocorrección y reordenamiento de resultados**. Sitúa las técnicas según el momento en que actúan:

- **Antes de buscar:** reformular la consulta, descomponer preguntas compuestas, multi-query o HyDE.
- **Al buscar:** top-k amplio, filtros por metadatos y búsqueda híbrida léxica + vectorial.

- **Después de buscar:** reordenar y evaluar si la evidencia basta; Self-RAG lleva esa crítica al propio modelo.
- **Al generar:** contexto sin duplicados, rechazar si no consta y citas verificables.

Lo resaltado en menta es lo que combina el CRAG de S11. Cada técnica añade coste y hay que medirla.

Diapositiva 6 · CRAG mete un juicio entre recuperar y generar

Recorre los cuatro pasos y su nodo: recuperar (`retrieve`, hasta 10 candidatos), reordenar (`rerank`, `[0, 1]`), juzgar (`assess`, cobertura) y corregir (`retain` / `discard` → `reformulate`) o responder.

- **Qué no es:** no es GraphRAG, no entrena el modelo y no garantiza mejorar cada respuesta. Hoy lo comprobaremos con datos.
- **Alcance:** inspirado en Yan, Gu, Zhu y Ling (arXiv:2401.15884). El artículo usa un evaluador ligero, búsqueda web y descomposición del conocimiento; nosotros adoptamos la idea de evaluar y corregir sobre un corpus local.

Diapositiva 7 · Mismo proyecto, otra política de recuperación

Compara en la tabla:

- **Búsquedas:** 1 búsqueda (`top-4`) en RAG básico frente a hasta **3 búsquedas (`top-10`)** y **2 reformulaciones** en CRAG.
- **Control:** un único umbral de similitud frente a un **juez** que puntúa cada fragmento en `[0, 1]` y estima la cobertura de la pregunta. En el laboratorio el juez es el mismo modelo que genera (`claude-sonnet-5-5`); los umbrales que deciden la ruta están en código.
- **Contexto:** lo recuperado una vez frente a conservar, unir sin duplicados o descartar.
- **Coste:** más llamadas y más latencia.

Corpus, índice, embeddings, modelo generativo e instrucciones son iguales en los dos; solo cambia la política de corrección.

Diapositiva 8 · El grafo de LangGraph, nodo a nodo

Abre `lab/src/crag-graph.ts` (y `apps/api/src/crag-graph.ts`) junto al diagrama:

- `retrieve` → `rerank` → `assess`; las aristas condicionales (`addConditionalEdges`) deciden la ruta.
- **Suficiente** → `answer`, que responde citando solo fragmentos aceptados en el orden validado.
- **Parcial** → `retain`; **insuficiente** → `discard`. Ambas pasan por `reformulate`, que vuelve a `retrieve` con la nueva `currentQuery`.
- **Límite:** con 3 búsquedas agotadas, o si la reformulación repite una consulta previa, el grafo termina en `refuse` **sin llamar al modelo generativo**.
- **Juez real del laboratorio:** `apps/api/src/lab-models.ts` (`reranker`, `assessor`, `reformulator`). Un id omitido o inventado por el juez es un error, no se rellena.

Diapositiva 9 · Relevancia de un fragmento ≠ cobertura de la pregunta

Muestra los umbrales de `CRAG_SCORE_THRESHOLDS` (`packages/rag/src/crag.ts`) y `LAB_CRAG_CONFIG` (`lab/src/config.ts`):

- **`minChunkRelevance = 0,50`:** un fragmento entra en la evidencia útil (máximo 6) si su puntuación es al menos `0,50`.
- **`partialCoverageMin = 0,40`:** por debajo, o sin ningún fragmento útil → **`insufficient`**.
- Entre `0,40` y `0,75`, o con lagunas → **`partial`**.
- **`sufficientCoverageMin = 0,75`** sin lagunas → **`sufficient`**.

El ejemplo de la diapositiva es el del guion local (S5 con `0,80`, cobertura `0,50`). En el laboratorio, `decideLabSufficiency` (`lab/src/reranker.ts`) aplica estos mismos umbrales a la valoración del juez. Una puntuación no es una probabilidad de acierto.

Diapositiva 10 · Parcial conserva; insuficiente descarta

Recorre las dos columnas con los guiones que se ejecutarán en local:

1. **Parcial (`partial-s5-s6`):** S5 (`0,800`) es útil, pero falta observabilidad (cobertura `0,50`); `retain` conserva S5, `reformulate` busca «observabilidad Langfuse y LangSmith de un agente», la búsqueda 2 trae S6 (`0,860`) y `answer` cita las dos sin duplicar.
2. **Insuficiente (`irrelevant-then-s6`):** S16 (`0,200 < 0,50`) se descarta con motivo e intento; se reformula desde la pregunta original y S6 (`0,880`) responde. S16 nunca entra en el prompt.

Avisa: en el test real de Zocomar **ninguna respondible necesitó la ruta parcial**; la reformulación solo apareció en las preguntas sin respuesta. Las rutas existen y se enseñan con los guiones, pero no siempre se activan.

Diapositiva 11 · La pregunta original nunca se sobrescribe

Con el extracto de `Annotation.Root` de `lab/src/crag-graph.ts`:

- `originalQuestion` no cambia y es la referencia de `assess` y `answer`; `currentQuery` es lo que recibe el retriever.
- `queries` guarda las consultas ejecutadas: una consulta repetida corta el bucle.
- `retrievalCount` (máx. 3) y `reformulationCount` (máx. 2) hacen cumplir el límite; `retained` acumula la evidencia útil sin duplicados y `discarded` guarda id, motivo e intento.
- Cada turno nace vacío: contadores a 0 y sin checkpointer.

Diapositiva 12 · Los límites viven en el código, no en el prompt

- **3** búsquedas (incluida la primera; un fallo también cuenta), **2** reformulaciones como máximo, sin reintentos ocultos (`maxRetries` a 0).
- **Plazo único por turno en el asistente:** la diapositiva dice 20 s, que es el valor de la demo local y del contrato por defecto. El asistente publicado con Sonnet usa **60 s**. En el laboratorio cada llamada tiene 30 s.
- **0** llamadas al generador cuando el turno termina en rechazo.
- Cuatro salidas que no se mezclan: `answer`, `refuse` («no consta»), `clarify` (calendario sin grupo, antes de buscar) y `error` (que nunca cuenta como rechazo correcto).

20–75 min · Demo en vivo (Diapositiva 13)

La diapositiva 13 lista los seis guiones y nombra el laboratorio AetherCloud con `bun run lab:demo`. Úsala como índice de la primera parte; la segunda parte es el laboratorio **Zocomar** con modelos reales.

Parte 1 · Seis guiones del grafo (unos 20 min, sin coste)

En local (`bun run dev`, `http://localhost:5173/`, etapa **CRAG**). Los guiones solo existen en modo demo; en la web publicada no aparece el selector.

1. **Suficiente a la primera** (`sufficient-s5`): recuperar, reordenar, valorar y responder.
2. **Evidencia parcial y segunda búsqueda** (`partial-s5-s6`).
3. **Primera evidencia descartada** (`irrelevant-then-s6`).
4. **Dato ausente y rechazo** (`missing-info`): exactamente **3 búsquedas**, **2 reformulaciones** y `refuse` con **0 llamadas al generador**.
5. **Calendario sin cohorte** (`clarify-cohort`): `clarify` con 0 búsquedas.
6. **Fallo controlado del evaluador** (`evaluator-error`): `error`, sin reintentos.

En el panel **Traza de la clase** señala los pasos `retrieve`, `rank` y `assess`, la consulta original frente a la reformulada y los conservados y descartados. Los guiones enseñan el recorrido; **no miden calidad**.

Parte 2 · Zocomar: RAG frente a CRAG con modelos reales (unos 35 min)

Presenta el corpus (2–3 min). **Zocomar** es un marketplace inventado: 12 políticas en `lab/corpus/sources/` (devoluciones, electrónica y garantía, envíos vigentes, envíos de 2025 sustituidos, pagos, vendedores, disputas, Plus, productos restringidos, atención al cliente y promociones), 54 fragmentos, PostgreSQL + pgvector `vector(768)` con HNSW, Gemini Embedding 2 y Sonnet. Las políticas reproducen fallos típicos de una documentación real:

- **Remite a otro documento:** devoluciones dice que el plazo de abono está en la política de pagos.
- **Excepción por categoría:** electrónica abierta, 14 días; auriculares intraauditivos sin precinto, sin devolución.
- **Política sustituida:** envíos de 2025 (gratis desde 29 €) frente a la vigente (39 € por vendedor).
- **Lenguaje del cliente:** «me cobrasteis dos veces» frente a «cargo duplicado».

Qué no metemos en el corpus: **pedidos y stock**. Cambian cada segundo y son de cada cliente; se consultan con SQL o con una herramienta, no con búsqueda semántica.

Comprobación previa sin coste:

```
docker compose -f lab/docker-compose.yml up -d
bun run lab:real status
```

Debe mostrar 54 vectores de 768 dimensiones y el índice HNSW. Después, en la misma terminal, una pregunta en las dos etapas:

```
bun run lab:real compare --question '¿Qué descuento tienen los estudiantes
universitarios en Zocomar?' --execute --max-spend-usd 5 --brief
```

El tope de 5 USD es el del diario `lab/.local/zocomar`, autorizado el 24 de octubre de 2026; tras el ensayo hay 2,7741628 USD reservados. Cada comparación reserva entre 0,05 y 0,2 USD (más en las preguntas sin respuesta, que agotan tres búsquedas).

Casos recomendados, con lo que ocurrió en el ensayo real (`docs/evidence/zocomar-2026-10-24/`):

Caso	Qué enseña	RAG en el ensayo	CRAG en el ensayo
Estudiantes universitarios (sin respuesta)	El umbral, por los pelos	Similitud 0,724 sobre un umbral de 0,7226: llama al generador, que dice «no consta» pero cita el precio de Plus	Relevancia ≤ 0,10 en tres búsquedas; rechaza sin generar
¿A partir de qué importe es gratis el envío?	Contexto limpio	Entran en el contexto dos fragmentos de la política de 2025; Sonnet los descarta por su cuenta	El juez da 0,10 al de 2025; solo entra la vigente
Reembolso de un pedido pagado con Bizum	Dos documentos	Responde bien: los dos fragmentos están en el top-4	Responde igual en una búsqueda
Cascos de botón estrenados	Excepción por categoría	Responde bien	Responde igual en una búsqueda

En cada caso señala: vecinos con su **similitud**, **relevancia del juez**, ruta y las dos respuestas. Dilo claramente: **en acierto suelen empatar**; la diferencia está en el margen y en lo que llega al generador.

Si falla la red o el presupuesto, abre los recibos guardados (`docs/evidence/zocomar-2026-10-24/demo-*.json`) o ejecuta el respaldo `bun run lab:demo`, y di qué estás proyectando.

75–90 min · Métricas, resultados reales, ejercicio y dudas (Diapositivas 14 a 16)

Diapositiva 14 · Qué medir en un sistema RAG

Tercera pieza del temario: **evaluación y métricas específicas de sistemas RAG**. Cada familia de métricas mira un punto de fallo distinto:

- **Recuperación (¿llega la evidencia?):** Recall@k, MRR, nDCG y precisión del contexto.
- **Generación (¿la respuesta se apoya en ella?):** acierto frente a la referencia (`X/14`), revisión manual de fuentes y fidelidad.
- **Abstención (¿sabe decir «no consta»?):** rechazos correctos (`Y/6`), falsos rechazos y datos inventados.
- **Operación (¿cuánto cuesta corregir?):** búsquedas y reformulaciones medias, latencia, tokens y coste conocido.

Añade dos lecturas que hoy resultan decisivas: el **margen de decisión** (cuánto separa la señal las preguntas con respuesta de las que no la tienen) y el **contexto que llega al generador** (fragmentos obsoletos o irrelevantes).

Protocolo: desarrollo → validación (calibrar) → congelar → test de **20 preguntas nuevas (14 + 6)**. Ajustar después de ver el test lo convierte en validación. En Zocomar, las dos correcciones del juez se hicieron con los casos de demostración, **antes** de abrir el test.

Diapositiva 15 · Resultados (proyecta los de Zocomar)

La diapositiva muestra el 16/20 → 17/20 del **respaldo determinista** de AetherCloud, que no mide modelos reales. **Proyecta en su lugar** `docs/evidence/zocomar-lab-2026-10-24.md` y explica:

Etapa	Respondibles	Rechazos correctos	Búsquedas medias	Mediana con respuesta	Mediana sin respuesta
RAG básico	**14/14**	**6/6**	1,00	3,2 s	0,5 s
CRAG	**14/14**	**6/6**	1,60	8,9 s	21 s

1. **Acierto: empate.** Revisión manual de las 40 respuestas. Ninguna respondible necesitó una segunda búsqueda en CRAG.
2. **Margen:** en RAG, la respondible más débil sacó 0,7309 y la mejor sin respuesta 0,7166: **0,014** alrededor del umbral 0,7226. Con las similitudes guardadas, un umbral de 0,70 deja pasar tres preguntas sin respuesta al generador y uno de 0,74 pierde dos respondibles. El juez dio ≥ 0,95 al documento correcto y ≤ 0,30 a lo que no responde: **0,65** de margen.
3. **Contexto limpio:** RAG pasó la política de envíos de 2025, ya sustituida, al generador en **3 de 14** respuestas; Sonnet la descartó por su cuenta. El juez le dio ≤ 0,10 y CRAG nunca la pasó.
4. **Coste:** latencia mediana ×2,8 en las respondibles y ×40 en las que no tienen respuesta; por tokens, el juez es casi todo el coste añadido (unos 0,66 USD estimados en todo el test, sin factura conciliada).

Conclusión para el alumno: con este corpus y este generador, **CRAG no mejora el acierto; compra margen y contexto limpio a cambio de latencia**. Un generador más débil o un corpus con más ruido podrían cambiar el resultado; no se ha medido. Veinte preguntas son una muestra pequeña: no es una tasa general.

Si preguntan por el lote histórico del asistente del 3 de octubre (17/20 → 18/20), es de una configuración anterior y otro corpus; no se mezcla con Zocomar.

Diapositiva 16 · Construir un CRAG completo desde cero

Abre `material/ejercicios/S11-enunciado.md` (dedicación orientativa: 6 a 8 horas):

- Proyecto **reconstruible desde 8 o más documentos ficticios e índice vacío** en PostgreSQL/pgvector (`vector(768)` + HNSW), con modelos configurados por variables de entorno y **conservando el RAG básico de S10** como línea base. Cada alumno elige su dominio; una tienda con sus políticas funciona bien.
- Grafo en **LangGraph** con pasos separados: `retrieve` (hasta 10 candidatos) → `rerank` (escala `[0, 1]`) → `assess` (umbrales calibrados) → `retain` / `discard` → `reformulate` (máximo 2 reformulaciones, 3 búsquedas) → `answer` / `refuse`.
- Trazas de las tres rutas, de un rechazo y del límite.
- **Comparación ejecutada sobre 20 preguntas nuevas (14 + 6)** con `X/14`, `Y/6`, recuperaciones medias, latencia y revisión manual de fuentes. Recomendado: el margen de decisión y el contexto que llega al generador.
- No se exige Ragas, interfaz gráfica, búsqueda web ni que CRAG gane. Un empate bien medido es un resultado válido. Material de apoyo: `lab/` y `material/ejercicios/laboratorio-apoyo.md`.
- Cierra abriendo el turno de dudas hasta las 20:00. El QR ya se mostró al inicio.